# Ders 25 Çalışma Özeti

## Ders 25 Çalışma Özeti

### Genel Konular

- Hocanın kolu ağrıdığı için kod yazmak yerine ağırlıklı olarak teorik konular anlatılır:
  - Dizilerin bellekteki yerleşimi ve dinamik bellek.
  - Sıralı diziye ekleme, kaydırma maliyeti ve insertion sort.
  - Ardışık yer bulma sorunu ve bağlı listeler (linked list).
  - Matris çarpımı ve çarpım sırası.
  - Belleğe sığmayan veriyi sıralama (dış sıralama).
  - Alt küme toplamı ve gezgin satıcı problemi gibi zor problemler, sezgisel (heuristic) ve açgözlü (greedy) yaklaşımlar.
  - Projede analizin nasıl yapılacağı.
  - Ürün üçlüleri ve görüntü işleme örnekleri Cuma gününe kaldı.
- Veri yapıları ve diziler
  - Veri yapıları, veriyi ihtiyaca göre en uygun şekilde saklamak ve hızlı erişmek içindir. Üç temel algoritmik yapı (sıralı, şartlı, tekrarlı) her zaman yetmez, belleği organize etmek gerekir. En basit veri yapısı dizidir.
  - `int A[3][4][5]`, beş tane $3 \times 4$ matrisin üst üste konmuş hâli gibi düşünülür. Programda dizi kaç boyutlu olursa olsun bellek tek boyutludur. Elemanlar bellekte ardışık (sequentially located) durur. Dizinin adı ilk elemanın (`A[0]`, `A[0][0]`, `A[0][0][0]`) adresini gösterir.
  - Rastgele erişim (random access): `A[6][18]` gibi bir elemana bir adres hesabıyla doğrudan ulaşılır.
  - Sıralı erişim (sequential access): ilk elemandan başlayıp pointer'ı her adımda tip boyutu kadar (`sizeof`) ilerletmek. 100 öğrencinin notlarını sırayla girmek veya okumak gibi işlerde kullanılır.
  - Dizinin boyutu genellikle statiktir (`#define` veya sabit sayı). Ama okunacak verinin boyutu çoğu zaman program çalışmadan bilinmez. Örneğin önceki derste okunan resmin satır ve sütun sayısı resmin başlığından, çalışma anında okunmuştu. Bunun için dinamik bellek gerekir. `scanf` ile `n` okuyup `int D[n]` yazılamaz.
  - Dinamik bellek işletim sisteminden izinle alınır. Temel fonksiyonlar `malloc`, `calloc`, `realloc` ve `free`'dir. Gelecek dönem Yapısal Programlama'da görülecek. Dinamik de olsa statik de olsa dizi bellekte ardışık yerleşir.
- Sıralı diziye ekleme ve silme
  - Sıralı bir diziye (4 6 8 12) yeni eleman eklemek için önce son elemana bakılır. Yeni eleman sondan büyükse doğrudan sona eklenir ve son indis bir artırılır.
  - Araya eklenecekse (ör. 7) baştan yer aranır, sonra o yerden sona kadar bütün elemanlar bir sağa kaydırılır (shift): `j` son indisten `i`'ye kadar `A[j+1] = A[j]; j--;`. Ardından eleman `A[i]`'ye yazılır ve son indis güncellenir.
  - Kaydırma en kötü durumda $n$ işlemdir. Silmede de elemanlar sola kaydırılır. Tek bir ekleme veya silme için $n$ işlem maliyetli bir durumdur.
- Insertion sort
  - Günlük hayatta kart dizerken yapılan iş: bir kartı alıp önceki kartlar arasında yerini bulmak ve araya koymak (insert).
  - Bubble sort yan yana elemanları tek tek takas eder ve gereksiz takaslar yapar (4 3 2'de önce 4-3, sonra 4-2, sonra 3-2). Insertion sort bu gereksiz takasları azaltır.
  - Algoritma ikinci elemandan (`i = 1`) başlar. Elemanı `key`'e yedekler, `j = i - 1`'den geriye doğru `j >= 0 && arr[j] > key` olduğu sürece `arr[j+1] = arr[j]; j--;` ile

kaydırır, sonunda `arr[j+1] = key` yazar. Her adımdan sonra `i`'ye kadar olan kısım sıralıdır.
  - 4 3 2 10 12 üzerinde adım adım izlendi: 3 4 2 10 12, sonra 2 3 4 10 12. Sonra 1 ve 5 eklenerek 1 2 3 4 5 10 12 elde edildi.
  - Insertion sort, derste anlatıldığı için proje önerisi olarak kabul edilmedi.
- Ardışık yer bulma sorunu
  - Dizilerin iki dezavantajı vardır: ekleme ve silmede kaydırma maliyeti, ve ardışık yer zorunluluğu.
  - Bellekte toplam 1,5 milyon boş hücre olabilir, ama bunlar bitişik (contiguous) değilse (500 bin, 300 bin, 200 bin gibi parçalar) 1 milyonluk bir dizi ayrılamaz. Statik veya dinamik fark etmez.
- Bağlı listeler (linked list)
  - Her düğüm (node) bir veri ve bir sonraki düğümün adresini (next) tutar. Düğümlerin bellekte ardışık olması gerekmez. Biraz fazladan bellek (next alanı) karşılığında esneklik kazanılır.
  - Benzetme: herkese tek tek mesaj göndermek yerine hoca mesajı bir öğrenciye, o bir sonrakine gönderir. En sondaki öğrenci kimseye göndermez. İlk düğüme head denir. Son düğümün next alanında `NULL` vardır.
  - Uzunluğu dinamik olarak değişir. Araya ekleme ve silmede kaydırma gerekmez, yalnızca bağlantılar değiştirilir. Ama rastgele erişim yoktur. Bir elemana ulaşmak için head'den başlayıp $n$ adım gidilir. Listeyi gezmek için head'in adresini bilmek gerekir, sona gelindiği `NULL`'dan anlaşılır.
  - Dizi ile bağlı liste arasında bir ödünleşim (trade-off) vardır: dizide ek bellek yok ve rastgele erişim var, ama ekleme-silme pahalı ve ardışık yer gerekir. Bağlı listede ek next alanı ve sıralı erişim var, ama ekleme-silme ucuz ve yer bitişik olmak zorunda değil.
  - Düğümde tek bir sayı değil, bir struct (ör. bir polinomun katsayıları) de saklanabilir.
- Dizilerle bağlı liste (ders notundaki örnek)
  - Gerçekte bağlı listeler struct ve pointer'larla yazılır. Bu derste mantığı göstermek için iki dizi ve bir head değişkeni kullanılır: veri dizisi `4 3 2 1 5`, link dizisi ve `head = 3`. Sıra: 3. hücre (1), 2. hücre (2), 1. hücre (3), 0. hücre (4), 4. hücre (5), link -1 (son).
  - Ekleme: yeni veri dizinin boş bir hücresine yazılır. Önceki düğümün link'i yeni hücreyi, yeni hücrenin link'i önceki düğümün eski hedefini gösterecek şekilde güncellenir. Hiçbir eleman kaydırılmaz.
  - Silme: head'den başlayıp silinecek veri aranır. Aranırken önceki düğümün indisi de tutulur. Önceki düğümün link'i, silinen düğümün link'ini gösterecek şekilde değiştirilir. Silinen hücre bellekte kalır. Hoca yeniden kullanılabileceğini belirtmek için o hücrenin link'ine -2 gibi özel bir değer yazmayı önerir.
  - Sıralı ekleme yapılacaksa doğru yeri bulmak için yine $n$ adımlık arama gerekir. Ama kaydırma maliyetinden kurtulunur.
  - Ders notundaki kod incelendi: elemanlar sırayla eklenip sıralı tutuluyor, `sil` fonksiyonu elemanı bağlantılardan çıkarıyor ve liste head'den başlanarak sırayla yazdırılıyor.
- Diğer bağlı liste türleri
  - Çift yönlü bağlı liste: her düğümde next ve previous alanları var. Örnek: bir dilekçenin hiyerarşide yukarı gidip onaylanarak aynı yoldan geri dönmesi. İki yönde de gezilebilir.
  - Dairesel bağlı liste: son düğümün next'i `NULL` yerine head'i gösterir.
  - C'de düğüm `struct node { int data; struct node *next; };` biçiminde tanımlanır. Bir struct kendi türünden bir alanı doğrudan içeremez, ancak ona pointer tutabilir. Java, C#, C++'ta referanslar bunu kolaylaştırır.
- Matris çarpımı
  - $A$ ($n \times p$) ile $B$ ($p \times m$) çarpılabilmesi için $A$'nın sütun sayısı $B$'nin satır sayısına eşit olmalıdır. Sonuç $n \times m$'dir.
  - `for i: for j: { sum = 0; for k: sum += A[i][k] * B[k][j]; C[i][j] = sum; }`. Karmaşıklık $n \cdot p \cdot m$, kare matrislerde $n^3$.
  - Robot kolu gibi uygulamalarda eklem hareketleri art arda matris çarpımlarıyla hesaplanır. Bu yüzden çarpımı az maliyetle yapmak önemlidir.
  - Çarpım sırası: $A$ (10 × 30), $B$ (30 × 5), $C$ (5 × 60). $(AB)C$ için $10 \cdot 30 \cdot 5 + 10 \cdot 5 \cdot 60 = 1500 + 3000 = 4500$ çarpma. $A(BC)$ için $30 \cdot 5 \cdot 60 + 10 \cdot 30 \cdot 60 = 9000 + 18000 =$

$27000$ çarpma. Sonuç aynı, maliyet çok farklıdır. Bu optimizasyon ilerideki derslerde görülecek.

- Belleğe sığmayan veriyi sıralama (dış sıralama)
  - Örnek: 9 GB veri, 1 GB RAM. Veri RAM'e sığacak parçalara bölünür. Her parça RAM'e alınıp quick sort gibi bir algoritmayla sıralanır ve diske geri yazılır.
  - Sonra her parçanın başından 100 MB'lık bölümler RAM'e alınır (toplam yaklaşık 900 MB). Bunlar merge mantığıyla birleştirilir ve sonuç diske yazılır. Bir parçanın bölümü bitince o parçadan yenisi okunur.
- Zor problemler
  - Alt küme toplamı: 4 8 3 6 5 7 dizisinde toplamı 10 olan bir alt küme var mı? Bütün alt kümeleri denemek $2^n$ işlemdir. Bir laboratuvar sorusunda (3 butonla ampul) bunun sabit 3 elemanlı bir hâli çözülmüştü.
  - Gezgin satıcı problemi: bütün şehirleri bir kez gezip başlangıca dönen en kısa rotayı bulmak. Bütün rotaları denemek $n!$'dir. Plaka sırasıyla gezmek gibi körü körüne bir yöntem çok uzun sürer.
  - Açgözlü yaklaşım: her şehirden en yakın gidilmemiş şehre gitmek (en yakın komşu). Çok daha hızlıdır ama en iyi rotayı garanti etmez. Derste bir örnekle başlangıç noktasına göre daha kötü bir rota çıktığı gösterildi.
  - Karınca kolonisi, parçacık sürü optimizasyonu ve arı algoritması gibi sezgisel arama algoritmaları iyi ama kesin olmayan çözümler bulur. Ölçütler: süre, bellek, rotanın uzunluğu. Bulunan bir çözüm daha iyisi bulununca güncellenir.
- Projede analiz
  - Karmaşıklık iki açıdan incelenir: çalışma zamanı ve bellek.
  - Çalışma zamanı için saniye ölçmek (`clock`) belirleyici değildir. Temel işlemin bulunduğu yere bir sayaç (counter) koyup kaç kez çalıştığı sayılmalıdır. Takas (swap) değil, kontrol (karşılaştırma) temel işlem olarak alınır.
  - Sonuçlar yıldızlarla çizilen basit bir histogramla gösterilebilir (`n = 10, 1000, 4000` için ayrı ayrı).
  - Bellek için kaç elemanlık dizi kullanıldığı raporlanabilir (ör. 100 eleman için 600 elemanlık yer).
  - Diffie-Hellman projesi üzerine: üs alma daha maliyetli bir işlemdir ve temel işlem olarak alınabilir. Hazır bir üs alma fonksiyonu kullanılıyorsa iç maliyeti bilinmez. Kendi yazılan döngülü veya hızlı üs alma ile karşılaştırılabilir.
  - Hoca, analiz kısmının notun en az %30'unu etkileyeceğini söyler, çünkü dersin amacı budur.
  - Ertesi Çarşamba sayısal analiz, Cuma bu dersin son kısmı.

## Hocanın Özellikle Vurguladığı Kısımlar

- Bellek tek boyutludur. Çok boyutlu diziler bellekte ardışık yerleşir.
- Dizide ekleme ve silme kaydırma yüzünden $n$ işlemdir.
- Bağlı liste ile dizi arasında bir ödünleşim vardır. Hangisinin kullanılacağı probleme bağlıdır.
- Zor problemlerde kaba kuvvet ($2^n$, $n!$) yerine sezgisel yöntemler kullanılır, ama bunlar en iyi sonucu garanti etmez.
- Projede analiz için temel işlemi sayın, saniye ölçmekle yetinmeyin.

## Kısa Tekrar Notları

- Dizi: ardışık, rastgele erişim $O(1)$, ekleme ve silme $O(n)$.
- Bağlı liste: düğüm = veri + next. Head ile başlar, `NULL` ile biter. Erişim $O(n)$, bağlantı değiştirerek ekleme ve silme.
- Insertion sort: `key = arr[i]`, büyükleri sağa kaydır, `arr[j+1] = key`.
- Matris çarpımı $n \cdot p \cdot m$. Çarpım sırası maliyeti değiştirir.
- Alt küme toplamı $2^n$, gezgin satıcı $n!$.

## Detaylı Açıklamalar

### Ders sonrası notlar

- **Dinamik belleğin neden gerektiği:** Hoca dinamik belleğin çok yer kapladığı için özel izin gerektirdiğini söyledi. Asıl fark yerin nereden alındığıdır. Yerel diziler stack'te durur ve stack küçüktür (Windows'ta varsayılan 1 MB). Bu yüzden `int resim[500][500];` gibi büyük bir yerel dizi programı çökertebilir. `malloc` ile alınan yer heap'tendir ve çok daha büyük olabilir. Boyut çalışma anında belirlenebilir.
- **`int D[n]` hakkında:** C89'da derleme sırasında bilinmeyen boyutla dizi tanımlanamaz. C99'daki değişken uzunluklu diziler (VLA) bunu stack üzerinde yapar, ama aynı stack sınırı geçerlidir (bkz. Ders 12 notları). Bu derste C89 kuralları geçerlidir.
- **Silinen hücrenin yeniden kullanılması:** Dizi tabanlı bağlı listede boşalan hücreler kendi aralarında ikinci bir listede (boş hücre listesi, free list) tutulursa yeni eklemelerde boş hücre aramaya gerek kalmaz. Hocanın önerdiği -2 işareti de aynı amaca hizmet eder, ama boş hücreyi bulmak için dizinin taranması gerekir.
- **Alt küme toplamı için daha iyi yöntem:** Sayılar pozitif tam sayılarsa, problem dinamik programlamayla $n \cdot k$ adımda çözülür ($k$ hedef toplam). Bu, $2^n$'den çok daha iyidir ama $k$ çok büyükse yine pahalıdır. Genel durumda problem NP-tamdır.
- **Gezgin satıcıda bütün rotalar:** Başlangıç şehri sabitlenip rotanın iki yönü aynı sayılırsa farklı rota sayısı $\frac{(n-1)!}{2}$'dir. Mertebesi yine $n!$'dir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.